FinanceFlow

Personal Finance Management Application

A Modern Full-Stack Web Application

Web Programming Project

Node.js | Express.js | MySQL/PostgreSQL | Vanilla JavaScript

April 2026

Agenda

1. Project Overview
2. **Front-End** — What & Why
3. **Back-End** — What & Why
4. **Database** — What & Why
5. System Architecture
6. Database Schema
7. API Design
8. Key Features

Project Overview

FinanceFlow is a full-stack personal finance app for tracking income, managing expenses, setting savings goals, and gaining financial insights.

Core Features

- Real-time dashboard with charts
- Income & expense CRUD
- Savings goals with progress
- Analytics & health score

Highlights

- Multi-currency (INR, USD, EUR, GBP)
- Dark/light mode
- Responsive (mobile-first)
- Zero-config startup

Front-End Technology

What is used?

Technology	Version	Role
Vanilla JavaScript	ES6+	Application logic
Tailwind CSS	CDN (latest)	Utility-first styling
Chart.js	CDN (latest)	Financial visualizations
Material Symbols	CDN	Icons
Google Fonts (Inter)	CDN	Typography

Architecture

Multi-Page Application (MPA) — each page is a self-contained HTML file with embedded JS. No build tools, no bundlers, no `node_modules` on the frontend.

Front-End — Why These Choices?

Why Vanilla JS?

- **Zero build step** — no bundlers
- **No dependencies** — instant load
- **Simpler debugging** — no abstractions
- **Direct deployment** — static files
- Modern ES6+: async/await, modules

Why Tailwind CSS?

- **Utility-first** — rapid UI dev
- **Responsive** built-in (sm/md/lg)
- **Dark mode** — class-based toggle
- Plugins: `forms`, `container-queries`

Why Chart.js?

- Simple API for charts
- Interactive tooltips
- Responsive & lightweight

Back-End Technology

What is used?

Technology	Version	Role
Node.js	18+	Server runtime
Express.js	v5.2	Web framework / REST API
TypeScript	v5.6 (partial)	Type-safe services & routes
Helmet	v8.1	Security HTTP headers
CORS	v2.8	Cross-origin access
express-rate-limit	v8.3	API rate limiting
compression	v1.8	gzip response compression
dotenv	v16.6	Environment configuration

Back-End — Why These Choices?

Why Node.js?

- **JS everywhere** — same language
- **Non-blocking I/O** — concurrent
- **npm ecosystem** — largest registry
- **Vercel-native** — serverless ready
- ES Modules throughout

Why Express.js v5?

- **Minimal & flexible** — unopinionated
- **Industry standard** — huge community
- **Middleware pipeline** — composable
- **Easy REST APIs** — simple routes
- v5: async error handling

Security Stack

Database Technology

What is used?

Technology	Driver	Role
MySQL 8	mysql2 v3.20	Local/self-hosted database
PostgreSQL	pg v8.16	Cloud database (Neon)
In-Memory (JS)	—	Zero-config fallback

Auto-Detection

The `connection.mjs` module inspects `DATABASE_URL`:

- `postgres://...` → PostgreSQL mode
- `mysql://...` or discrete `DB_*` vars → MySQL mode
- No config → In-memory fallback

Database — Why These Choices?

Why MySQL?

- Widely documented
- ACID compliant
- InnoDB: FK support
- Easy local setup

Why PostgreSQL?

- Cloud-native (Neon)
- Production-grade
- Free tier
- Advanced features

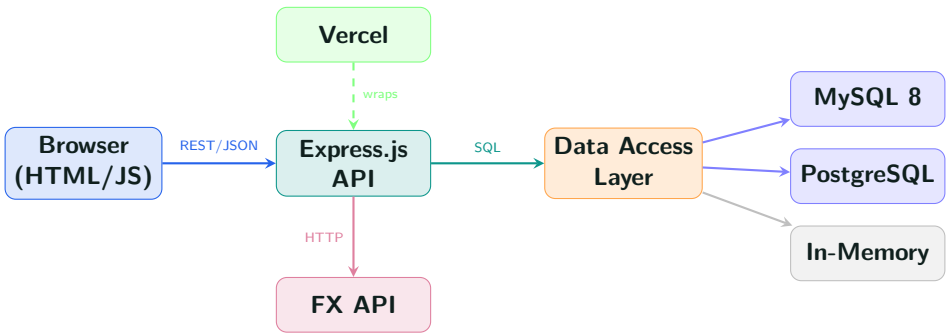
Why No ORM?

- Full SQL control
- Prevents injection
- No hidden magic
- Zero overhead

Why Dual Support?

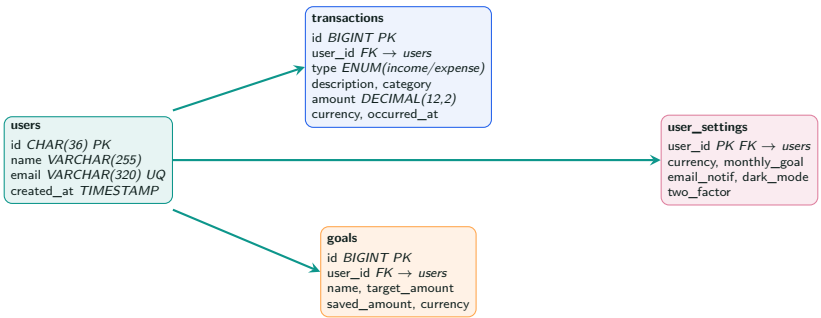
Flexibility — MySQL for local dev, PostgreSQL for cloud. No lock-in.

System Architecture



Database Schema

4 core tables with foreign key relationships



All foreign keys use **ON DELETE CASCADE** for referential integrity.

REST API Endpoints

Method	Endpoint	Description
Authentication		
POST	/api/auth/login	Login
POST	/api/auth/signup	Register
Dashboard & Analytics		
GET	/api/dashboard	Dashboard stats, goals, recent txns
GET	/api/analytics	Monthly trends, category breakdown
Transactions		
GET	/api/transactions	List (filter: type, category, search)
POST	/api/transactions	Create transaction
PUT	/api/transactions/:id	Update transaction
DELETE	/api/transactions/:id	Delete transaction
Goals & Settings		
GET/POST/PUT/DELETE	/api/goals	Full CRUD for savings goals
GET/PUT	/api/settings	User preferences
GET	/api/fx/latest	FX rates (6h cached)

Key Features (1/2)

Dashboard

- Total income, expenses, balance
- Savings rate percentage
- Financial Health Score (0–100)
- Interactive charts

Income & Expenses

- Full CRUD operations
- Category filtering & search
- Pagination (10/page)

Key Features (2/2)

Analytics

- Weekly/monthly trends
- Category breakdown charts
- Income vs expense comparison

User Experience

- Dark/light mode toggle
- Responsive (mobile-first)
- Multi-currency support
- Toast notifications

Financial Health Score

Dynamic scoring algorithm

The dashboard computes a **Financial Health Score (0–100)** based on three weighted factors:

Factor	Weight	What it measures
Savings Rate	40%	(Income − Expenses)/Income
Expense Consistency	30%	Stability of monthly spending
Income Growth	30%	MoM income change

Score Ranges

- **80–100**: Excellent
- **60–79**: Good

Security Features

Threat	Mitigation
SQL Injection	Parameterized queries (`?` / `$1` binding)
XSS / Clickjacking	Helmet.js (CSP, X-Frame-Options, HSTS)
Brute Force	express-rate-limit
Data Leaks	Stack traces hidden in production
Orphaned Data	FK constraints with ON DELETE CASCADE
Bandwidth Abuse	gzip compression
Cross-Origin	CORS middleware

Data Integrity

All user data is scoped by `user_id` foreign keys. Input validation on all API endpoints. DECIMAL(12,2) ensures precise financial calculations.

Deployment

Vercel Serverless + Neon PostgreSQL

How It Works

1. Browser → Vercel edge network
2. Vercel routes to `api/index.js` serverless function
3. Express processes via middleware pipeline
4. Data access layer queries Neon PostgreSQL
5. JSON response returned

Configuration

- `vercel.json` rewrites all routes to API handler
- `DATABASE_URL` for Neon PostgreSQL
- Auto-migration on first connection
- SSL enabled for cloud DB

Project Structure

```
financeflow/
|-- financeflow/                # Frontend (Vanilla JS + Tailwind)
|   |-- dashboard/             # Dashboard page
|   |-- income_page/           # Income tracking
|   |-- expenses_page/         # Expense management
|   |-- reports_analytics/     # Analytics + Chart.js
|   |-- settings/              # User settings
|   |-- auth/                  # Login & Signup
|   +-- shared/                # financeflow.js, darkmode.js, toast.js
|
|-- server/                     # Backend (Node.js + Express)
|   |-- src/
|   |   |-- full-server.mjs    # Main Express server
|   |   |-- data-access.mjs    # Hybrid data layer (MySQL/PG/Memory)
|   |   |-- db/                # connection, queries, migrations
|   |   |-- routes/            # API route handlers
|   |   +-- services/          # Business logic (TypeScript)
|   +-- migrations/001_init.sql
|
|-- api/index.js                # Vercel serverless wrapper
+-- vercel.json                 # Deployment config
```

Technology Stack Summary

Layer	Technology	Purpose
Frontend	Vanilla JavaScript (ES6+)	App logic
	Tailwind CSS (CDN)	Responsive styling
	Chart.js	Financial charts
	Material Symbols	Icons
Backend	Node.js 18+	Runtime
	Express.js v5	REST API framework
	TypeScript (partial)	Type safety
	Helmet + CORS + Rate Limit	Security
Database	MySQL 8	Local/self-hosted DB
	PostgreSQL (Neon)	Cloud production DB
	In-Memory fallback	Zero-config dev mode
DevOps	Vercel	Serverless hosting
	open.er-api.com	FX rates API

Thank You

Questions?

FinanceFlow — Personal Finance Management Application

Built with Node.js | Express.js | MySQL/PostgreSQL | Vanilla JS